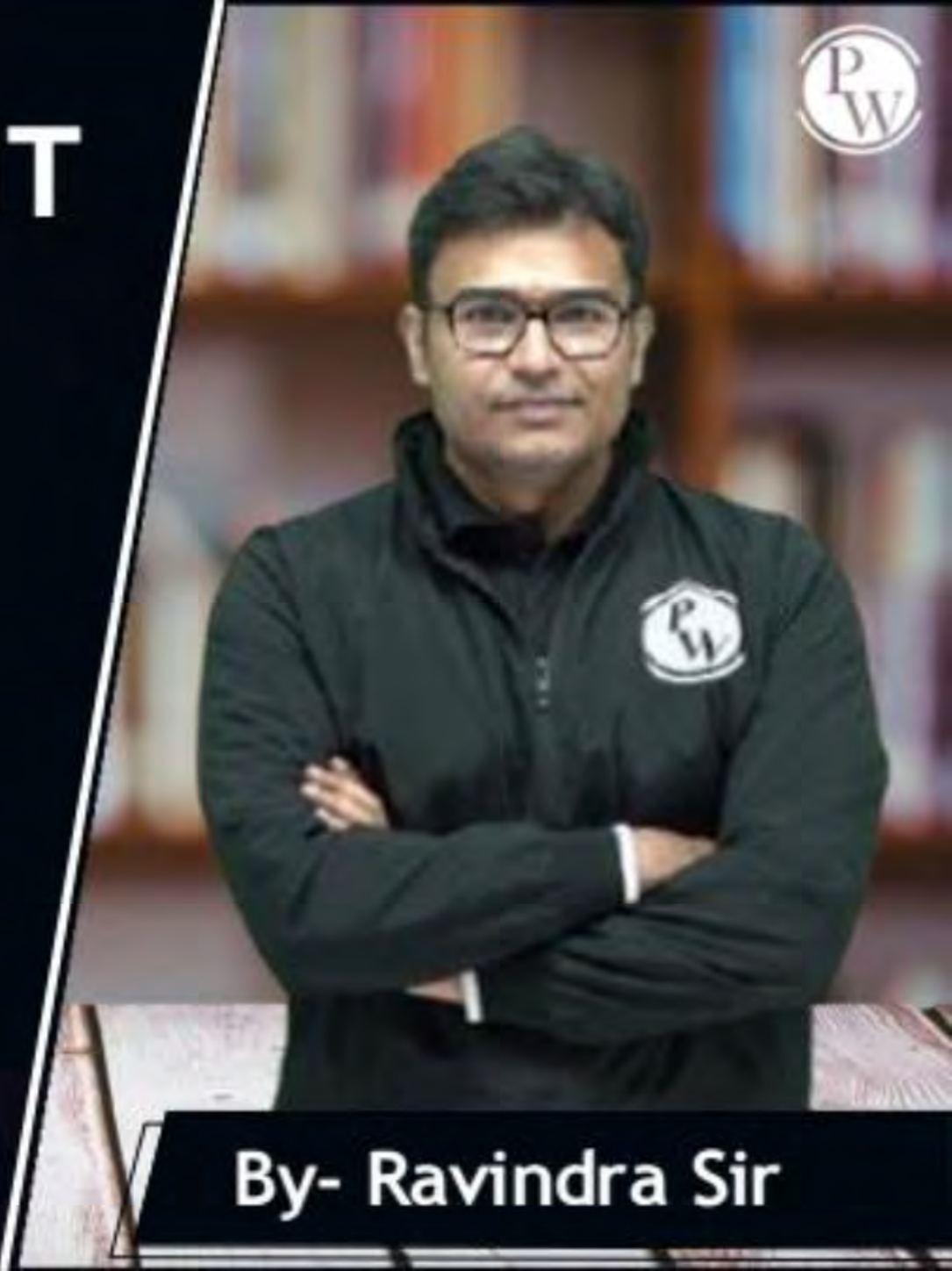
Computer Science & IT

COMPILER DESIGN
(CD)

SDT

Lecture No. 2



By- Ravindra Sir

Recap of Previous Lecture



- Topic
- Topic
- Topic

SDT

Topics to be Covered



- Topic
- Topic
- Topic

SDT

Join Telegram

@RBRSIRPW





Inspiring Stories : Mula Gabharu

Mula Gabharu – The Queen Who Raised Women Soldiers

Background: Mula Gabharu was the daughter of an Ahom king in Assam in the early 1500s.

Struggles: Her husband fell fighting an invading army. Her people were helpless and losing hope.

Achievements: She trained women warriors, led them into battle with courage, and confronted the enemy herself before being martyred.

Impact: If she could turn grief into action and lead women into battle, you can turn doubt into study and lead your rank up.



Inspiring Stories : B. N. Jagatap

B. N. Jagatap

Background: Grew up in a modest family in rural Maharashtra with no strong science background at home.

Education: PhD in Nuclear Magnetic Resonance (NMR) from IISc Bangalore.

Career Achievements: Improved NMR methods to study molecules in great detail, helping drug design and materials research.

Impact: Made high-end science tools more effective for Indian research labs and universities.



Inspiring Stories : Veeramangai Kuyili

Veeramangai Kuyili – The Woman Who Became a Human Torch

Background: Kuyili was a trusted soldier of queen Velu Nachiyar from Tamil Nadu in the 1700s.

Struggles: She worked without praise and lived in a time when women were rarely seen in battle.

Achievements: During one attack, she covered her body in ghee, set herself on fire, and jumped into the British armory. Her act became a key turning point in the war.

Impact: If she could light herself on fire to save her queen, you can light up that answer sheet with focus



Inspiring Stories : K. J. Rao

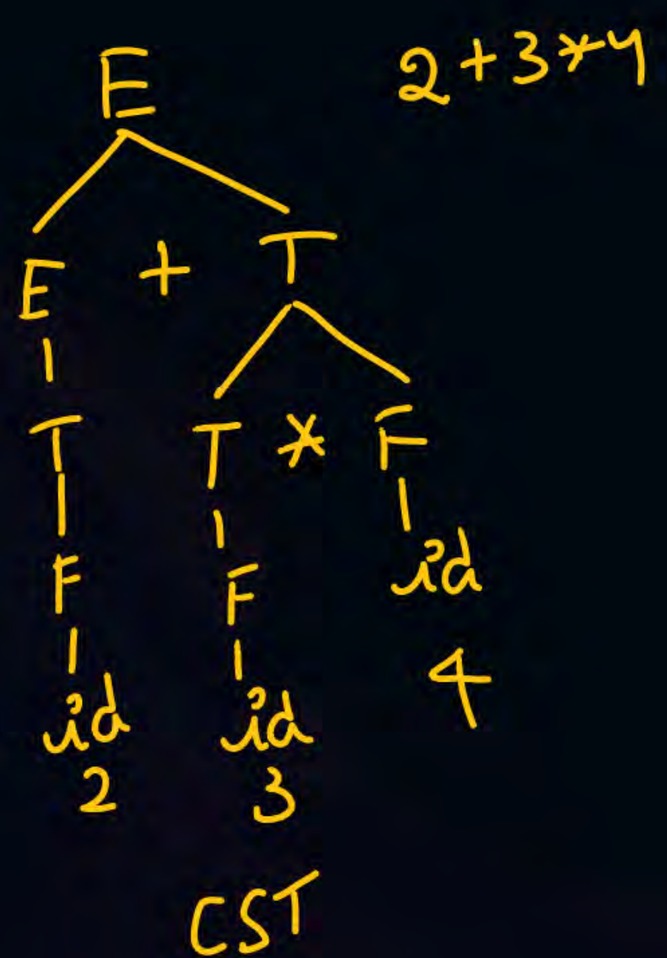
K. J. Rao

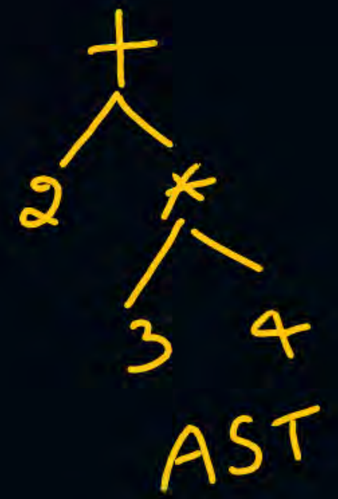
Background: Born in a small town in Karnataka with very limited means. Had to work hard to afford studies.

Education: PhD in Solid State Chemistry from IISc Bangalore.

Career Achievements: Developed new ceramic materials that can store energy and work in extreme heat.

Impact: Helped India make better energy storage systems and heat-resistant tools for industry.


E
2+3*4
E
+
T
T
F
id
2
T
*
F
F
id
3
id
4
CST


+
2
*
3
4
AST

The parser produces a tree while checking the Syntax of the programming language construct. This is called parse tree. A parse tree gives the complete Syntax behind the string. Hence it is called Concrete Syntax tree whereas a condensed form of a parse tree is called Syntax tree which abstracts away unnecessary terminal and non terminals Hence it is called abstract Syntax tree (AST)

$E \rightarrow E_1 + T \quad \{ E.nptr = mknode(E_1.nptr, '+', T.nptr) \}$

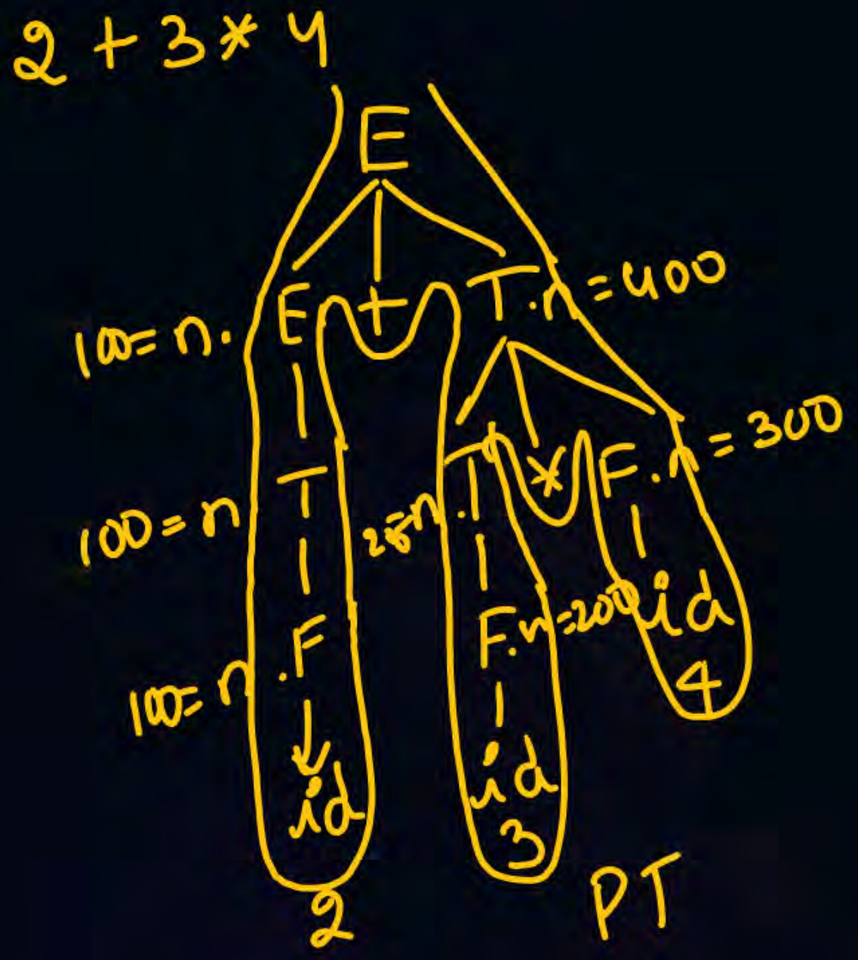
$/T \quad \{ E.nptr = T.nptr \}$

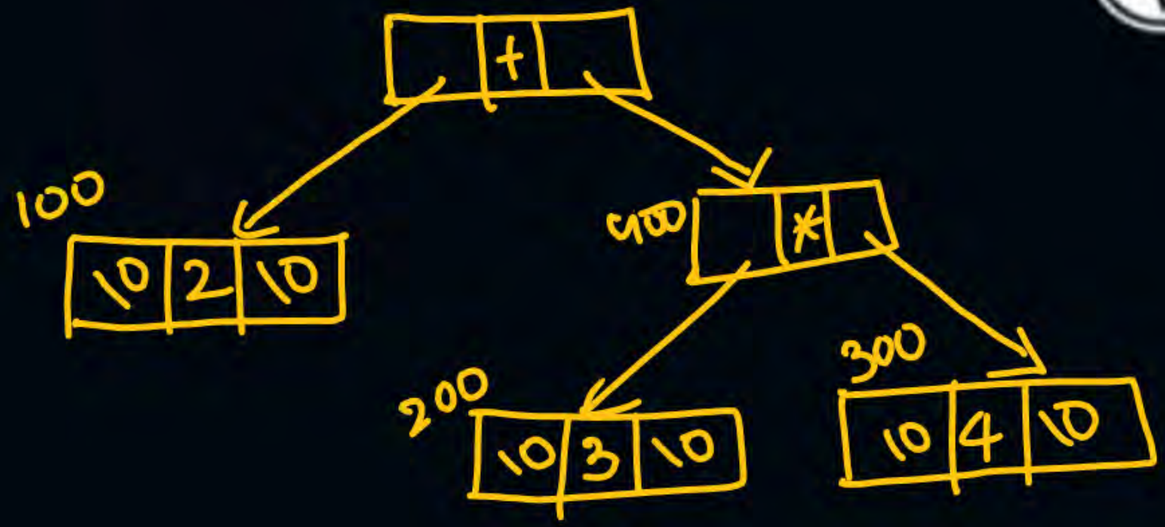
$T \rightarrow T_1 * F \quad \{ T.nptr = mknode(T_1.nptr, '*', F.nptr) \}$

$/F \quad \{ T.nptr = F.nptr \}$

$F \rightarrow id \quad \{ F.nptr = mknode(null, id.nptr, null) \}$

SDT to Construct AST

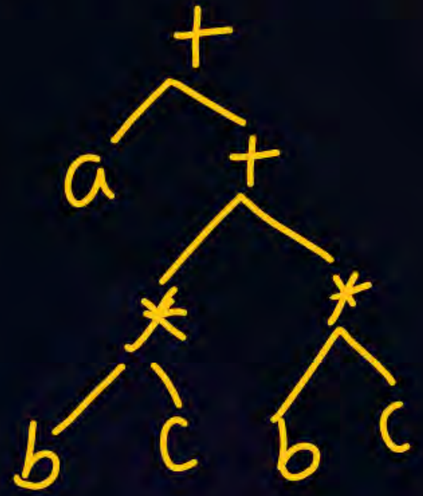

2 + 3 * 4
E
E
+
T
100 = n.
T.n = 400
100 = n
2 = n
T
*
F
F.n = 300
100 = n
F
F.n = 200
id
4
id
id
2
3
PT

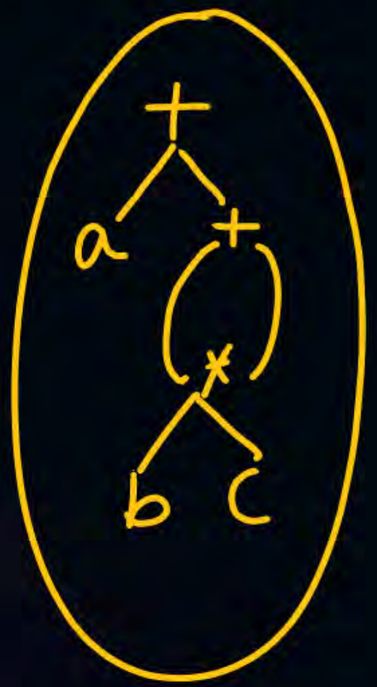

+
100
10
2
10
400
*
200
10
3
10
300
10
4
10
AST

Directed acyclic graph :-

DAG is used to identify Common Sub expressions. It helps in eliminating redundant code

Ex :- a + (b * c) + (b * c)









Types of SDT :-

S attributed definition

i) It uses only Synthesized attributes

$A \rightarrow BCD$



L attributed definition

i) It uses both Synthesized and inherited attributes. But if a inherited attribute is present, then there is a restriction that each inherited attribute is restricted to inherit either from parent or left sibling only

S attributed definition

2) Semantic actions can be placed only at the RHS of a production

$A \rightarrow B \{ \}$

$A \rightarrow \{ \} B X$

3) Attributes are generally evaluated during bottom up parsing

L-attributed definition

2) Semantic actions can be placed any where on the RHS

3) Attributes are generally evaluated by traversing the parse tree depth first, left to right.

S-attributed definitions :-

All the examples we have seen so far are S-attributed definitions 'S' stand for Simple. It is also called postfix SDT, as semantic actions are placed only at the right end of the productions

Top down evaluation of S attributed definitions

So far we have seen bottom up evaluation of S attributed def. We can do top down evaluation also. In BU evaluation, whenever there is a reduction we performed the semantic action of the production. Where as with TDP we cannot tell when exactly the translations are carried out.

Given an SDT, a TDP carries out translations as follows:

1) Assumes Semantic actions are dummy non terminals which becomes part of RHS of production
2) Translations are pushed onto the stack along with grammar symbols.
3) When the dummy non terminals appears on the top of the stack the non terminal is popped and corresponding action is carried out

$A \rightarrow BC\{\ \} \Rightarrow A \rightarrow BC\#_1$ ✓



SDT for counting no of balanced parantheis

$S \to (S_1)$ { S.count = S_1.count + 1 }

$/\epsilon$ { S.count = 0 }

$S \to (S_1)\ \#_1$ $/\epsilon\ \ \#_2$ $\Rightarrow$ $S \to (S_1)\ \#_1$ (1), $S \to \#_2$ (2)

LL(1)

	(	)	$
S	$S \to (S)\ \#_1$	$S \to \#_2$	$S \to \#_2$

During the parsing, if #1 or #2 appears on top of the stack
the we will take out the symbol and carry out the translations

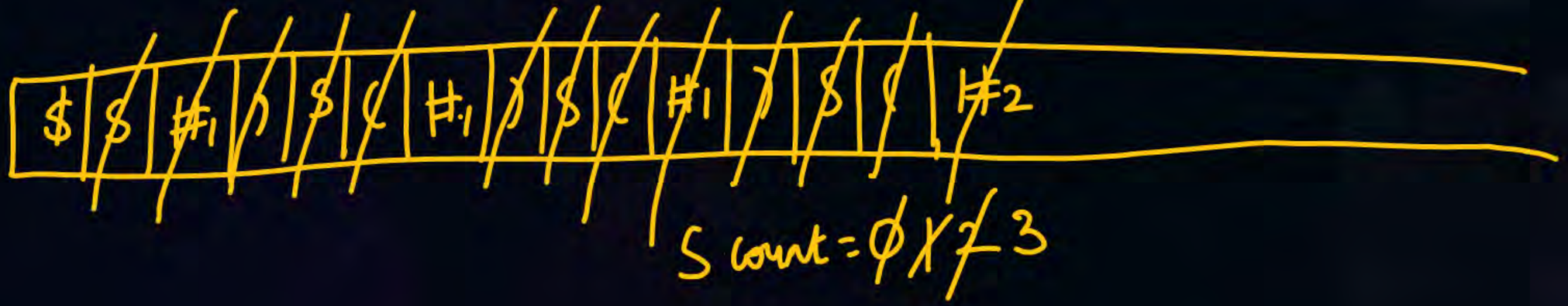
S - def → using TDP (LL(1))

LL(1) :-

	(	)	$
S	S→(S)#1	S→#2	S→#2



accepted



S count = 0 X ≠ 3

THANK - YOU